PostgreSQL

Ecosistema e fondamenti

Giorno 1 · mattina

Marco De Giovanni · 20 ottobre 2026

1.02 INTRODUZIONE

Come lavoriamo oggi

01

Parte A · SQL base

teoria: il linguaggio, SQL standard

→

02

Laboratorio · Fase 1

dal database legacy allo schema degli ordini

→

03

Parte B · PostgreSQL

teoria: struttura, motore, concorrenza

→

04

Laboratorio · Fase 2

il checkout, atomico e concorrente

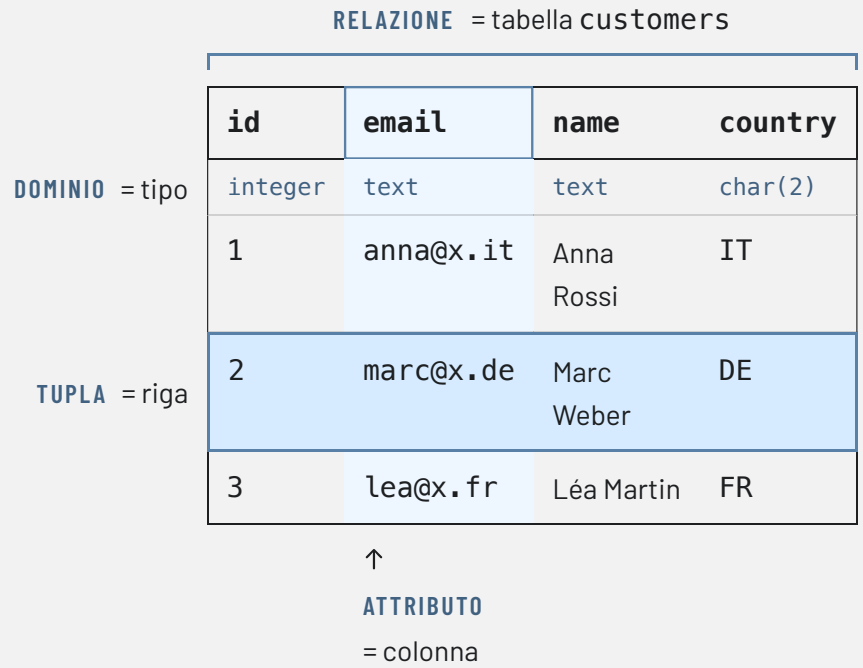
01

CICLO 1 · TEORIA

Parte A · SQL base

Standard SQL, valido su qualunque database relazionale. PostgreSQL entra in scena dal ciclo 2.

Modello relazionale, chiavi e integrità



- Una tabella è un **insieme non ordinato** di righe: senza `ORDER BY` nessun ordine è garantito.
- **Chiave candidata**: insieme minimo di colonne che identifica una riga. **Primary key**: la candidata scelta (unica e `NOT NULL`).
- **Foreign key**: colonne che puntano alla chiave di un'altra tabella → **integrità referenziale**.

Modello relazionale, chiavi e integrità

- I vincoli li fa rispettare il **database**, per qualunque client: i controlli solo applicativi non bastano.
- Prassi: chiave **surrogata** (id generato) + `UNIQUE` sulla chiave **naturale** (email, codice fiscale).

INTEGRITÀ REFERENZIALE

orders

ID	CUSTOMER_ID	STATUS
10	1	paid
11	1	shipped
12	2	pending
~~13~~	~~999~~	~~paid~~

FK → PK

customers

ID	EMAIL
1	c1@shop.it
2	c2@shop.it
3	c3@shop.it

■ Riga 13 **orfana**: il cliente 999 non esiste, la foreign key la rifiuta.

Funzioni SQL standard

FAMIGLIA	FUNZIONI	ESEMPIO
NULL	`COALESCE`, `NULLIF`	`COALESCE(phone, 'n.d.')`
testo	`UPPER`, `LOWER`, `TRIM`, `CHAR_LENGTH`, `SUBSTRING`	`UPPER(TRIM(email))`
numeri	`ROUND`, `ABS`, `CEIL`, `FLOOR`	`ROUND(price * 1.22, 2)`
date	`CURRENT_DATE`, `CURRENT_TIMESTAMP`, `EXTRACT`	`EXTRACT(YEAR FROM created_at)`
condizioni e tipi	`CASE WHEN`, `CAST`	`CAST(total AS numeric(10,2))`

Concatenazione standard: l'operatore a doppia barra verticale (`'a' || 'b'`).

DDL, DML e DQL: i tre sottolinguaggi di SQL

DDL

Data Definition Language

Definisce la **struttura**: tabelle, colonne, tipi, vincoli e indici.

`CREATE` `ALTER` `DROP`

ESEMPIO

```
ALTER TABLE customers
  ADD COLUMN phone text;
```

DML

Data Manipulation Language

Modifica i dati contenuti nelle tabelle: inserisce, aggiorna e cancella righe.

`INSERT` `UPDATE` `DELETE`

ESEMPIO

```
UPDATE orders
  SET status = 'paid'
  WHERE id = 10;
```

DQL

Data Query Language

Interroga i dati e restituisce un risultato, senza modificarli.

`SELECT`

ESEMPIO

```
SELECT email, country
  FROM customers
  WHERE country = 'IT';
```

DDL: tabelle e tipi

SQL

```sql
CREATE TABLE customers (
  id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email   text NOT NULL UNIQUE,
  country text NOT NULL
);
CREATE TABLE orders (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers (id) ON DELETE RESTRICT,
  status      text NOT NULL CHECK (status IN ('pending','paid','shipped','delivered','cancelled')),
  total       numeric(12,2) NOT NULL CHECK (total >= 0)
);
```

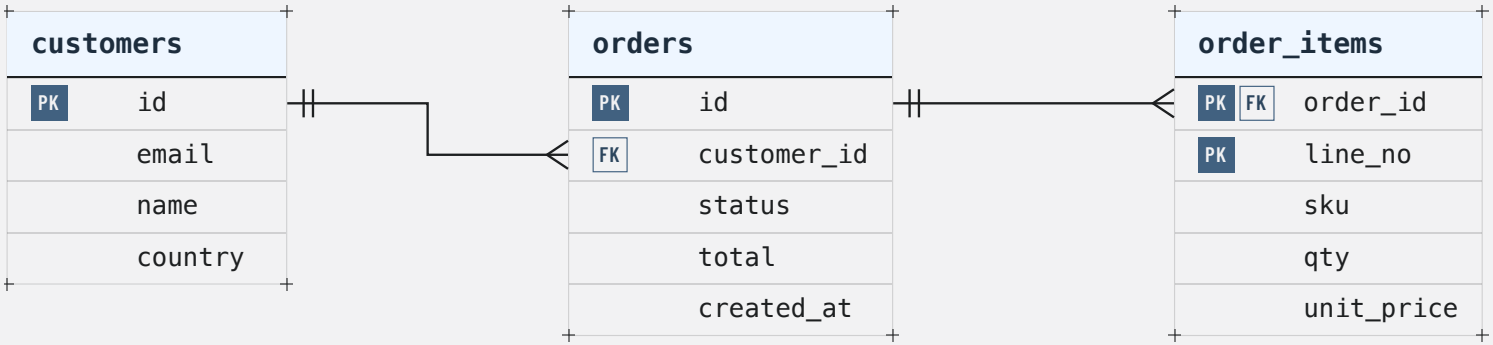
DDL: modificare ed eliminare

SQL

```sql
-- ALTER: modifica una struttura esistente
ALTER TABLE customers ADD COLUMN phone text;
ALTER TABLE customers ALTER COLUMN phone SET NOT NULL;
ALTER TABLE customers RENAME COLUMN phone TO mobile;
ALTER TABLE orders ADD CONSTRAINT orders_total_chk CHECK (total < 100000);
ALTER TABLE orders ADD CONSTRAINT orders_customer_fk
  FOREIGN KEY (customer_id) REFERENCES customers (id) ON DELETE RESTRICT;
ALTER TABLE orders DROP CONSTRAINT orders_customer_fk;
ALTER TABLE customers DROP COLUMN mobile;

-- DROP: elimina l'oggetto e i suoi dati
DROP TABLE IF EXISTS order_items;
DROP TABLE customers CASCADE;  -- elimina anche le FK che la referenziano
```

DDL: chiavi esterne e ON DELETE



- Tipi base: interi, **`numeric(p,s)` per il denaro (mai `float`)**, testo, data/ora, booleani.
- Nel corso: `orders → customers` **RESTRICT**, `order_items → orders` **CASCADE**.

DDL: i vincoli

VINCOLO	GARANTISCE
`PRIMARY KEY`	unicità e obbligatorietà
`UNIQUE`	unicità (ammette più `NULL`)
`NOT NULL`	valore obbligatorio
`CHECK`	regola di dominio (passa anche con `NULL`)
`FOREIGN KEY`	integrità referenziale; `ON DELETE RESTRICT`, `CASCADE`, `SET NULL`, `SET DEFAULT`

DQL: SQL è dichiarativo; l'ordine logico

- Descrivi **cosa** vuoi, non **come**: il **planner** sceglie algoritmo, ordine e indici in base alle statistiche.
- Si ragiona **per insiemi**: ogni clausola trasforma un insieme di righe in un altro.
- L'ordine logico è il contratto sul risultato: fisicamente il planner può riordinare.

ORDINE DI SCRITTURA

```
6 SELECT c.country,
    count(*) AS ordini,
    sum(o.total) AS fatturato
1 FROM orders o
1 JOIN customers c
    ON c.id = o.customer_id
2 WHERE o.status <> 'cancelled'
3 GROUP BY c.country
4 HAVING sum(o.total) > 1000000
8 ORDER BY fatturato DESC;
```

ORDINE LOGICO DI VALUTAZIONE



DQL: conseguenze dell'ordine logico

SCRITTURA	ESITO	PERCHÉ
alias del `SELECT` nel `WHERE`	■ errore	il `WHERE` viene prima
alias del `SELECT` nell'`ORDER BY`	□ funziona	l'`ORDER BY` viene dopo
`sum(...)` nel `WHERE`	■ errore	si usa `HAVING`
`GROUP BY` con alias	□ dipende dal DBMS (PostgreSQL lo accetta)	non standard
`LIMIT` senza `ORDER BY`	■ righe non deterministiche	nessun ordine garantito

DQL: SELECT, filtri, aggregazioni, sottoquery e CTE

SQL

```sql
SELECT c.country, count(*) AS ordini, sum(o.total) AS fatturato
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.status <> 'cancelled'
GROUP BY c.country
HAVING sum(o.total) > 1000000
ORDER BY fatturato DESC;
```

DQL: operatori insiemistici, `UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT`

- Combinano i risultati di due `SELECT` **in verticale** (righe); le JOIN, che vediamo tra poco, li combinano in orizzontale (colonne).
- Regola: stesso numero di colonne, tipi compatibili; i nomi vengono dalla prima query.

Regola pratica (es. `SELECT email FROM customers UNION ALL SELECT email FROM newsletter_subscribers`): usa `UNION ALL` di default; `UNION` solo se i duplicati vanno davvero eliminati.

OPERATORE	RESTITUISCE	DUPLICATI
`UNION ALL`	tutte le righe di A e di B	mantenuti, nessun costo extra
`UNION`	righe di A o di B	rimossi (ordina o fa hash: più lento)
`INTERSECT`	righe presenti in A e in B	rimossi
`EXCEPT`	righe di A assenti in B	rimossi

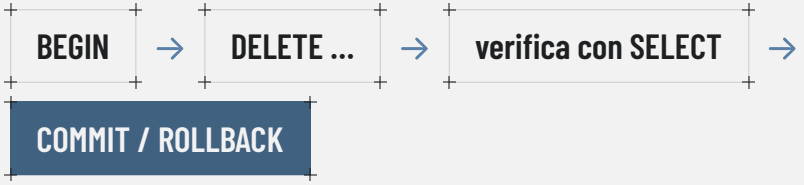
DML: INSERT, UPDATE, DELETE e transazioni

- `INSERT … VALUES (…), (…)` e `INSERT … SELECT`.
- `UPDATE … SET … WHERE …` e `DELETE FROM … WHERE …`: **senza WHERE toccano tutta la tabella**.
- `ON DELETE CASCADE` propaga la cancellazione ai figli; `RESTRICT` la blocca.
- Lavora **in transazione** quando non sei sicuro del risultato:

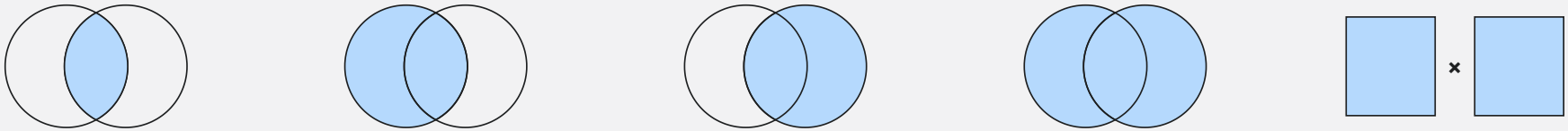
SQL

```sql
BEGIN;
DELETE FROM orders WHERE id = 10;   -- verifica con una SELECT
ROLLBACK;                           -- oppure COMMIT
```

LAVORARE IN SICUREZZA



JOIN



- Sempre `JOIN … ON` esplicito: un join senza condizione diventa un prodotto cartesiano.
- `USING (col)` è una scorciatoia; `NATURAL JOIN` è fragile (da evitare).

JOIN	RESTITUISCE
INNER	solo le righe con corrispondenza da entrambi i lati
LEFT	tutte le righe di sinistra; `NULL` a destra se manca la corrispondenza
RIGHT	come `LEFT` con le tabelle invertite
FULL	le righe senza corrispondenza da entrambi i lati (riconciliazioni)
CROSS	prodotto cartesiano N × M

Anti-join, semi-join e ON vs WHERE

- **Anti-join** ("clienti senza ordini"): `LEFT JOIN … WHERE o.id IS NULL` oppure `NOT EXISTS`.
- **Semi-join** ("clienti con almeno un ordine"): `EXISTS`, non duplica il cliente.
- Nei join esterni il **filtro sulla tabella di destra** cambia significato:

Filtro nel WHERE

■ diventa un INNER

SQL

```sql
SELECT c.name, o.id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.total > 60;
```

RISULTATO

NAME	ID
Rossi	10
Bianchi	12
~~Verdi~~	~~NULL~~

Filtro nella ON

□ tutti i clienti restano

SQL

```sql
SELECT c.name, o.id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
                  AND o.total > 60;
```

RISULTATO

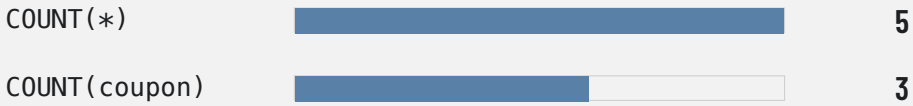
NAME	ID
Rossi	10
Bianchi	12
Verdi	NULL

NULL: logica a tre valori

- `NULL` = sconosciuto (non zero, non stringa vuota). `NULL = NULL` → `NULL`.
- Si usa `IS NULL`, `IS NOT NULL`, `IS DISTINCT FROM`.
- `WHERE` tiene solo le righe `TRUE`.
- Un `CHECK` è soddisfatto anche con `NULL` → si affianca `NOT NULL`.
- `COUNT(*)` conta le righe, `COUNT(col)` i valori non `NULL`; `SUM` di un insieme vuoto è `NULL` (`coalesce(sum(x), 0)`).

A	B	A AND B	A OR B
TRUE	NULL	NULL	TRUE
FALSE	NULL	FALSE	NULL

ESEMPIO · 5 ORDINI, 2 CON COUPON NULL



La trappola `NOT IN`

- Basta **un `NULL`** nella sottoquery e la condizione non è mai `TRUE`: **zero righe**, nessun errore.
- È un bug silenzioso: funziona finché la tabella non riceve il primo `NULL`.
- Soluzione: **`NOT EXISTS`**.

SQL

```sql
x NOT IN (1, 2, NULL)
-- equivale a
x <> 1 AND x <> 2 AND x <> NULL   -- l'ultimo termine è sempre NULL
```

NOT IN contro NOT EXISTS

NOT IN

■ 0 righe, nessun errore

SQL

```sql
SELECT name FROM customers
WHERE id NOT IN (
  SELECT customer_id FROM orders
);  -- contiene un NULL
```

RISULTATO

0 righe

NOT EXISTS

☐ risultato corretto

SQL

```sql
SELECT name FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.id
);
```

RISULTATO

NAME
Verdi

Normalizzazione: il problema

Una tabella "piatta", come la esporta un vecchio gestionale (è il punto di partenza del laboratorio):

	cliente, ripetuto su ogni riga d'ordine				prodotto, su ogni acquisto			
ORDINE_ID	CLIENTE_EMAIL	CLIENTE_NOME	CLIENTE_PAESE	RIGA	SKU	PRODOTTO	PREZZO	QTA
53	cliente72@shop.it	N. Cognome 72	IT	1	P11	Prodotto 11	38.00	1
53	cliente72@shop.it	N. Cognome 72	IT	2	P22	Prodotto 22	71.00	2
106	cliente143@shop.it	N. Cognome 143	ES	1	P10	Prodotto 10	35.00	1

Gli stessi fatti sono ripetuti su più righe: il cliente su ogni riga d'ordine, il prodotto su ogni acquisto. Ne nascono tre **anomalie**:

Regola: ogni fatto in un solo posto

Aggiornamento	il cliente 72 compare anche in altri ordini con il nome "Nome Cognome 72": correggerlo su una riga e dimenticare le altre lascia due nomi per la stessa persona.
Inserimento	non posso registrare un nuovo prodotto o cliente finché non c'è un ordine.
Cancellazione	cancellando l'unico ordine di un cliente perdo anche il cliente.

Le forme normali: la scala completa

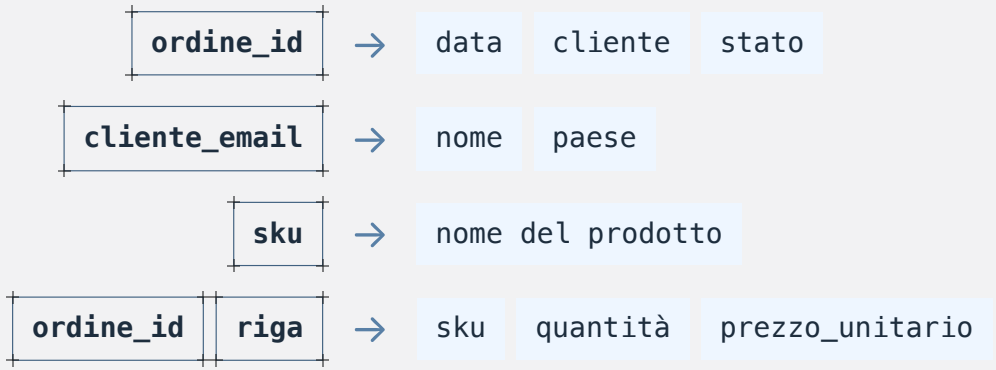
Perché ci fermiamo alla 3NF: è il livello di riferimento nella progettazione reale e copre il nostro caso. BCNF, 4NF e 5NF trattano casi rari (più chiavi candidate sovrapposte, attributi multivalore indipendenti, relazioni ternarie): uno schema in 3NF ben progettato di solito li soddisfa già.

FORMA	REGOLA	NEL CORSO
1NF	valori atomici, niente gruppi ripetuti	sì
2NF	1NF + ogni attributo dipende dall'**intera** chiave	sì
3NF	2NF + niente dipendenze **transitive** tra non-chiave	sì
BCNF	ogni determinante è una chiave candidata	no
4NF	BCNF + niente dipendenze **multivalore** indipendenti	no
5NF	4NF + niente dipendenze di **join** non implicate dalle chiavi	no

Forme normali: le dipendenze funzionali

Dipendenze funzionali ("A determina B"): la chiave della tabella piatta è `(ordine_id, riga)`.

DIPENDENZE FUNZIONALI · A DETERMINA B



1NF (valori atomici, niente gruppi ripetuti) è già rispettata: una riga per riga d'ordine. Il problema sono le dipendenze che **non partono dalla chiave intera**: le vediamo nelle prossime due slide.

Seconda forma normale, con un esempio

2NF — ogni attributo dipende dall'intera chiave.

Una dipendenza **parziale**: data e stato dipendono da `ordine_id` soltanto, non da `(ordine_id, riga)`.

PRIMA · NON NORMALIZZATA

righe_ordine

PK ordine_id	PK riga	data	stato	sku	qta
53	1	2026-03-02	paid	P11	1
53	2	2026-03-02	paid	P22	2
106	1	2026-03-05	shipped	P10	1

- ☐ Data e stato si ripetono su ogni riga dello stesso ordine: per spedire l'ordine 53 vanno aggiornate **due righe**, e basta dimenticarne una.

DOPO · 2NF

orders

PK id	data	stato
53	2026-03-02	paid
106	2026-03-05	shipped

order_items

PK FK order_id	PK line_no	sku	qta
53	1	P11	1
53	2	P22	2
106	1	P10	1

Nota: in un sistema e-commerce reale molto complesso, per supportare spedizioni parziali, la data e lo stato di spedizione potrebbero stare nella singola riga d'ordine. Ai fini didattici, la data dell'ordine è l'esempio perfetto di 2NF

Terza forma normale, con un esempio

3NF — niente dipendenze transitive tra attributi non-chiave.

Una dipendenza **transitiva**: `id` → `cliente_email` → nome e paese. Nome e paese non dipendono dall'ordine, ma dal cliente.

PRIMA · NON NORMALIZZATA

orders

PK id	cliente_email	cliente_nome	paese
53	cliente72@shop.it	N. Cognome 72	IT
71	cliente72@shop.it	**Nome Cognome 72**	IT
106	cliente143@shop.it	N. Cognome 143	ES

- ☐ Lo stesso cliente compare con **due nomi** negli ordini 53 e 71: nessuna regola tiene allineate le copie.

DOPO · 3NF

orders

PK id	FK customer_id
53	72
71	72
106	143

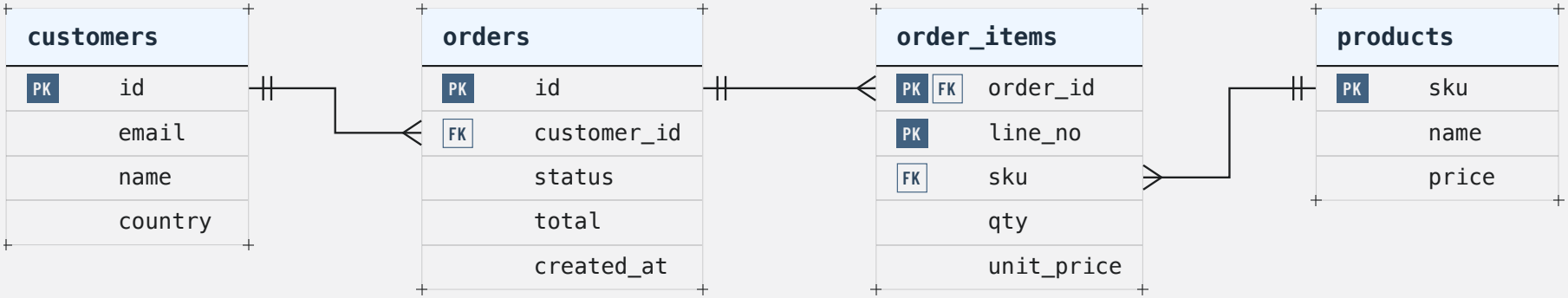
customers

PK id	email	name	country
72	cliente72@shop.it	N. Cognome 72	IT
143	cliente143@shop.it	N. Cognome 143	ES

Il nome si corregge in un solo posto. Lo stesso vale per `sku` → nome del prodotto: tabella `products`.

Forme normali: lo schema risultante

Risultato: `customers`, `products`, `orders`, `order_items`, collegate da foreign key.



+ `unit_price` **nella riga d'ordine non è una ridondanza**: è lo **snapshot storico** del prezzo pagato (il listino può cambiare).
+ Il **totale** dei prodotti è derivabile dalle righe: non si memorizza (vista), oppure lo si denormalizza **consapevolmente** nella tabella `orders` (dove potrebbe includere spese extra come spedizioni e tasse).
+ Normalizzare non è un dogma: si denormalizza per ragioni di prestazioni o storiche, ma **scegliendo**, non per caso.

I due dataset del corso

Fasi 1–2 (mattina): ShopIT, il caso "legacy"

OGGETTO	CONTENUTO
`legacy.ordini_flat`	una sola tabella piatta: 5.010 righe, 2.000 ordini, 300 clienti, 40 prodotti
schema `shop`	lo schema che **progetti tu**: `customers`, `products`, `orders`, `order_items`

I due dataset del corso

Fasi 3–4 (pomeriggio): il sito ShopIT in produzione

TABELLA	RIGHE	NOTE
customers	100.000	`email` univoca, `country`
orders	1.000.000	`status`, `total`, `created_at`, `tags text[]`, `attrs jsonb`
order_items	2.000.000	`sku`, `qty`, `unit_price`

02

CICLO 1 · PRATICA

Laboratorio

Fase 1 · Dal database legacy allo schema degli ordini

FO LABORATORIO · CICLO 1

Il caso e il setup

Il caso: **ShopIT**, un e-commerce, ha ereditato un database "legacy" con una sola tabella piatta degli ordini. Il team deve progettare la parte ordini del sito, dallo schema fino alla manutenzione.

Scorciatoie di DBeaver

Linux = Windows; macOS: ⌥ = Option, ⌘ = Command, ^ = Control

AZIONE	WINDOWS	LINUX	MACOS
Nuovo script SQL	`Ctrl+]`	`Ctrl+]`	`^]`
Esegui l'istruzione corrente	`Ctrl+Enter`	`Ctrl+Enter`	`^↩` / `⌘↩`
Esegui lo script	`Alt+X`	`Alt+X`	`⌥X`
Piano di esecuzione (Explain)	`Ctrl+Shift+E`	`Ctrl+Shift+E`	`^⇧E`
Commit / Rollback	`Ctrl+Alt+Shift+K` / `R`	`Ctrl+Alt+Shift+K` / `R`	`^4` / `^8`

F1 LABORATORIO · CICLO 1

Fase 1 · Dal database legacy allo schema degli ordini

01

Esplora

i dati e trova le anomalie.

02

Progetta

quattro tabelle vincolate, tipi, vincoli e `ON DELETE`.

03

Migra

`INSERT … SELECT` in transazione correggendo i dati sporchi.

04

Verifica

controlla conteggi e ricavi.

03

CICLO 2 · TEORIA

Parte B · PostgreSQL

Cosa SQL non dice: dal linguaggio al motore

SQL · IL COSA		POSTGRESQL · IL COME
Dove stanno fisicamente i dati?	2.03	in un **heap** di pagine da 8 kB; posizione `ctid`
Chi esegue la query e con quale memoria?	2.02	un **processo** per connessione (`work_mem`); `shared_buffers` condivisa
Quali tipi e quali costrutti oltre lo standard?	2.04, 2.05	`jsonb`, array, `uuid`, range; `RETURNING`, `ON CONFLICT`
Come sceglie come eseguire la query?	2.07	il **planner**, basato su costi e statistiche
Cosa succede con più utenti concorrenti?	2.08–2.15	**MVCC**, transazioni, isolamento, lock

PostgreSQL: il motore e il viaggio di una query

Processo

uno per ogni connessione

MVCC

nessuna lettura blocca una scrittura

WAL

durabilità delle modifiche

Planner

basato sui costi

Il viaggio di una query:

01

Il client apre una **connessione** → PostgreSQL crea un **processo backend**.

02

Il backend esegue **parser → rewriter → planner → executor**.

03

L'executor legge le pagine da `shared_buffers` (cache) o da disco.

04

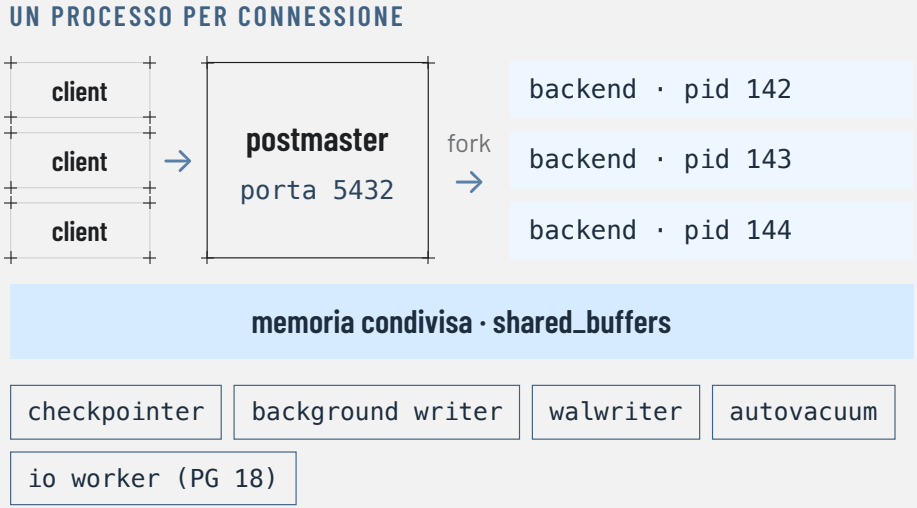
Una modifica crea una **nuova versione** di riga e viene scritta nel **WAL**.

05

Al `COMMIT` il WAL è durevole; le versioni vecchie le ripulisce il `VACUUM`.

Processi e memoria

- Il **postmaster** crea un **processo backend** per ogni connessione (non thread) → uso di pooler (PgBouncer, Agroal, HikariCP).
- **`shared_buffers`**: cache condivisa da 8 kB.

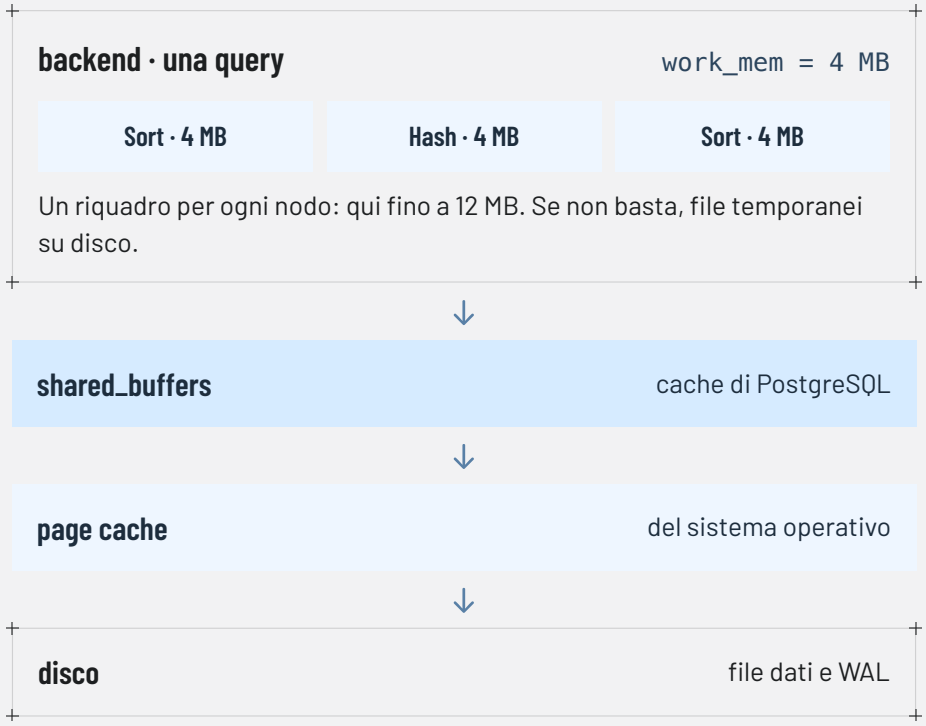


Processi di background, visibili in `pg_stat_activity`

Processi e memoria

- `work_mem` (4 MB): memoria allocata **per ogni nodo** di sort/hash della query.
- `maintenance_work_mem` (64 MB) per indici e vacuum.

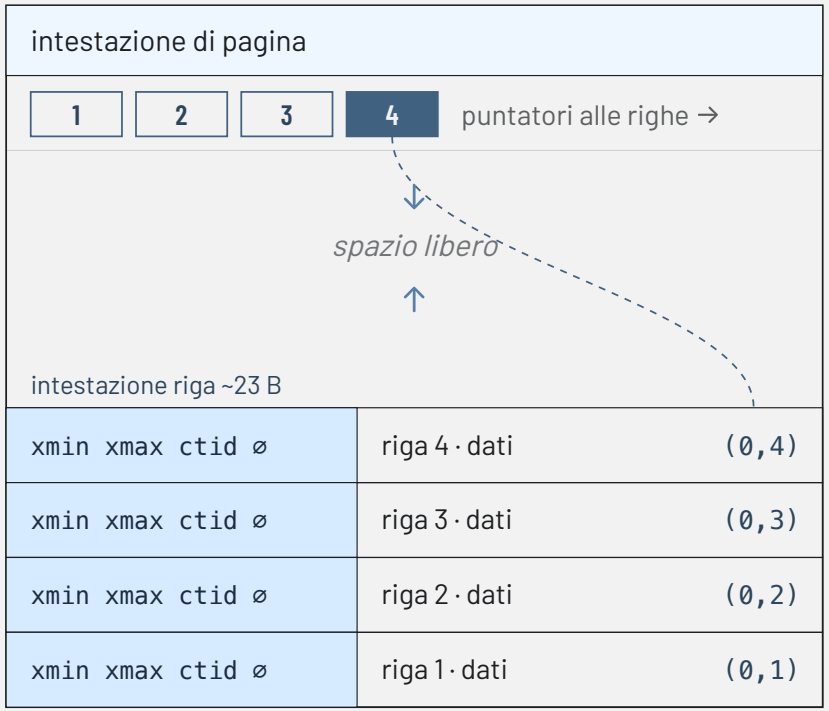
DOVE STA LA MEMORIA



Storage: heap, pagine e righe

+ Tabella = **heap** di pagine da 8 kB, **senza ordine logico garantito**.
+ `ctid` = `(pagina, posizione)`: cambia a ogni `UPDATE`.

PAGINA · 8 kB



`ctid = (pagina, posizione)` ⌀ = bitmap dei NULL

WAL: prima il log, poi i dati

- **WAL** (Write-Ahead Log): scrittura prima su file di log; durabilità e ripristino al crash.
- `UNLOGGED`: niente WAL, veloce, ma viene svuotata automaticamente in caso di chiusura anomala (crash).

WRITE-AHEAD LOG



Tipi di dato

USO	TIPO	NOTA
denaro	`numeric(p,s)`	mai `float`
istanti	`timestamptz`	memorizza istante UTC assoluto
testo	`text`	stesse prestazioni di `varchar(n)`
chiave	`bigint GENERATED ALWAYS AS IDENTITY`	standard; sostituisce `serial`
collezioni / json	`text[]`, `jsonb`, `uuid`	tipi estendibili

Il dialetto PostgreSQL

+ **`INSERT … RETURNING`**: restituisce id o valori generati subito.
+ **`ON CONFLICT … DO UPDATE`**: upsert atomico senza race conditions.
+ **`DISTINCT ON (col)`**: prima riga per gruppo.
+ **`count(*) FILTER (WHERE …)`**: aggregato condizionale nativo.

OBIETTIVO	SQL GENERICO	POSTGRESQL
id generato	seconda query dopo l'`INSERT`	`INSERT … RETURNING id`
upsert	`MERGE` o controllo applicativo	`ON CONFLICT … DO UPDATE`
prima riga per gruppo	`row_number() OVER (…)` + filtro	`DISTINCT ON (col)`
aggregato condizionale	`sum(CASE WHEN … END)`	`count(*) FILTER (WHERE …)`

Funzioni specifiche di PostgreSQL

FUNZIONE	COSA FA	ESEMPIO
`::`	cast abbreviato	`total::numeric(10,2)`
`now()`	istante di inizio transazione	`WHERE created_at > now() - interval '7 days'`
`date_trunc`	tronca una data all'unità	`date_trunc('month', created_at)`
`age`	intervallo tra due date	`age(now(), birth_date)`
`CONCAT`, `string_agg`	concatena valori o righe	`string_agg(name, ', ')`

Le funzioni standard viste in 1.04 restano valide: queste sono scorciatoie del dialetto.

DDL transazionale

- **DDL transazionale**: creazioni e modifiche tabelle annullabili con `ROLLBACK`.
- **Migrazioni atomiche**: più `CREATE` / `ALTER` nella stessa transazione si applicano tutti o nessuno.
- Dopo un **errore** la transazione è abortita: il `COMMIT` esegue in realtà un `ROLLBACK`.
- **Eccezioni**: `CREATE DATABASE`, `CREATE INDEX CONCURRENTLY` e `VACUUM` non possono stare in una transazione.

IL DDL È TRANSAZIONALE

```
BEGIN;                                          □ ok
CREATE TABLE prova (id int);                    □ ok
ROLLBACK;                                       ■ annullata
SELECT * FROM prova;                            ■ non esiste
```

UNA MIGRAZIONE CHE FALLISCE A METÀ

```
BEGIN;                                          □ ok
ALTER TABLE orders ADD COLUMN note text;        □ ok
CREATE INDEX ON orders (customer_idd);          ■ errore
COMMIT;                                         ■ esegue ROLLBACK
```

Schemi: lo spazio dei nomi

- **Gerarchia**: istanza (cluster) → **database** → **schema** → tabelle, viste, sequenze, funzioni.
- **Schema** = namespace dentro un database: lo stesso nome può esistere in schemi diversi (`legacy.ordini_flat`, `shop.orders`).
- **`public`** è lo schema creato di default: una tabella creata senza prefisso finisce lì.
- **`search_path`** decide in quali schemi cercare i nomi non qualificati (default: `"$user", public`).
- Nel corso: `SET search_path = shop, public;` oppure nomi sempre qualificati (`shop.orders`).

GERARCHIA DEGLI OGGETTI



Il planner: dal testo al piano

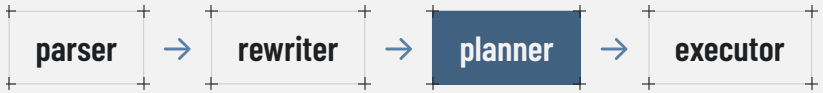
- Planner basato sui costi (I/O, CPU) tramite statistiche di `ANALYZE`.
- Ispezione del piano con `EXPLAIN`: un albero di nodi (`Seq Scan`, `Index Scan`, `Hash Join`...).
- `cost` = stima iniziale..totale in unità arbitrarie; `rows` = righe stimate.
- `EXPLAIN ANALYZE` esegue davvero la query e aggiunge tempi e righe reali.

SQL · PIANO

```sql
EXPLAIN SELECT * FROM orders
WHERE customer_id = 42;

-- QUERY PLAN
-- Seq Scan on orders
--   (cost=0.00..1834.00 rows=12 width=48)
--   Filter: (customer_id = 42)
```

PIPELINE NEL PROCESSO BACKEND



MVCC: versioni di riga

Multi-**V**ersion **C**oncurrency **C**ontrol: controllo della concorrenza tramite più versioni della stessa riga.

- Modifiche come **versioni di riga**: `UPDATE` = `DELETE` (vecchia riga con `xmax`) + `INSERT` (nuova riga con `xmin` e `ctid`).
- I lettori non bloccano mai gli scrittori e viceversa; **due scrittori sulla stessa riga** si bloccano.

CUSTOMERS · ID = 1, NELLO STESSO HEAP

XMIN	XMAX	CTID	NAME	
100	**101**	(0,1)	'Rossi'	MORTA
↓ UPDATE della transazione 101				
101	**102**	(0,4)	'Rossi R.'	MORTA
↓ UPDATE della transazione 102				
102	0	(0,7)	'R. Rossi'	VIVA

Ogni `UPDATE` imposta `xmax` sulla versione vecchia e ne scrive una nuova, con nuovo `xmin` e nuovo `ctid`.

Snapshot e visibilità

- **Read Committed**: nuovo snapshot preso a ogni singola istruzione.
- **Repeatable Read** / **Serializable**: snapshot fissato all'inizio della prima query.

LIVELLO	ISTRUZIONE 1	ISTRUZIONE 2	ISTRUZIONE 3
Read Committed default · uno snapshot per istruzione	SNAPSHOT SELECT …	SNAPSHOT SELECT …	SNAPSHOT SELECT …
Repeatable Read · Serializable uno snapshot alla prima query, poi fisso	SNAPSHOT SELECT …	STESSO SELECT …	STESSO SELECT …

Il prezzo dell'MVCC

- Versioni vecchie rimangono nell'heap come **dead tuples** (*bloat*).
- Bonifica demandata a **VACUUM** (autovacuum).

UNA PAGINA DOPO MOLTI UPDATE

viva	morta	morta	viva
morta	viva	morta	morta
viva	morta	viva	morta

Le righe morte (*dead tuples*) occupano spazio finché il VACUUM non le ripulisce.

transazione aperta a lungo
può ancora vedere le versioni vecchie

VACUUM fermo

Transazioni: ACID, errori, SAVEPOINT

- Qualsiasi errore SQL abortisce l'intera transazione (obbligo di ROLLBACK).
- I SAVEPOINT consentono di delimitare e annullare solo singole sezioni.

UN ERRORE DENTRO LA TRANSAZIONE

```
BEGIN;                          ☐ ok
INSERT INTO orders …;           ☐ ok
SAVEPOINT s;                    ■ savepoint
UPDATE stock …;                 ■ errore
SELECT …;                       ☐ abortita
ROLLBACK TO s;                  ■ ripristino
COMMIT;                         ☐ confermata
```

Livelli di isolamento e anomalie

+ Read Uncommitted promosso a **Read Committed** (nessuna Dirty Read possibile).
+ Conflitti di serializzazione falliscono con SQLSTATE `40001`.

■ anomalia possibile ☐ impedita ■ errore 40001: la transazione va ritentata

LIVELLO	DIRTY READ	NON-REPEATABLE READ	PHANTOM READ	ANOMALIA DI SERIALIZZAZIONE
Read Uncommitted → Read Committed	☐ mai	■ possibile	■ possibile	■ possibile
Read Committed	☐ mai	■ possibile	■ possibile	■ possibile
Repeatable Read	☐ mai	☐ mai	☐ mai	■ possibile
Serializable	☐ mai	☐ mai	☐ mai	■ mai (errore)

Lost update

+ Lost update evitato con calcolo atomico o `SELECT … FOR UPDATE`.

T	SESSIONE A · +30	SESSIONE B · +50
1	`SELECT saldo → 100`	
2		`SELECT saldo → 100`
3		`UPDATE saldo = 150; COMMIT`
4	**`UPDATE saldo = 130; COMMIT`**	la scrittura di B è persa

130

saldo finale: vince l'ultima scrittura

180

saldo atteso (100 + 30 + 50)

Write skew

+ Write skew intercettato e bloccato unicamente dal livello **Serializable**.

Regola: **almeno un medico di turno**. Sono di turno Alice e Bob.

T	TRANSAZIONE 1 · ALICE	TRANSAZIONE 2 · BOB
1	`SELECT count(*) … → 2`	`SELECT count(*) … → 2`
2	`UPDATE … Alice fuori turno`	`UPDATE … Bob fuori turno`
3	`COMMIT`	`COMMIT`

■ **Repeatable Read**: entrambe confermate, nessun medico di turno.

□ **Serializable**: una delle due abortisce con `40001`.

Lock espliciti e deadlock

+ `SELECT … FOR UPDATE SKIP LOCKED` per **code** con più consumatori performanti.
+ Prevenzione deadlock: accesso alle tabelle **sempre nello stesso ordine** logico.

ESEMPIO · UN WORKER PRELEVA IL PROSSIMO JOB

```
BEGIN;
SELECT id, payload FROM jobs
WHERE status = 'queued' ORDER BY id
LIMIT 1 FOR UPDATE SKIP LOCKED;
UPDATE jobs SET status = 'running' WHERE id = :id;
COMMIT;
```

CODA DI LAVORI · SKIP LOCKED



DEADLOCK · ATTESA CIRCOLARE



Retry nel codice

- Ritentare solo su errori transitori di concorrenza: `40001` (serializzazione) e `40P01` (deadlock).
- Lasciare email o API esterne fuori dal ciclo di retry transazionale.



Recap della mattina

01 Tabelle come insiemi con vincoli a carico del DB.

02 Ordine logico SQL, gestione `NULL` e `NOT IN`.

03 Processo per connessione, `work_mem`, storage heap e WAL.

04 Dialetto PG, DDL transazionale e indici FK manuali.

05 MVCC e pulizia dead tuples.

06 Livelli di isolamento e strategie di retry.

04

CICLO 2 · PRATICA

Laboratorio

Fase 2 · Il checkout: atomico e concorrente

F2 LABORATORIO · CICLO 2

Fase 2 · Il checkout: atomico e concorrente

01

Checkout atomico con vincoli stock e ripristino da `SAVEPOINT`.

02

Ispezione `ctid`, `xmin`, `xmax` e DDL transazionale.

03

Prevenzione overselling concorrente e code ordini con `SKIP LOCKED`.